

# SQL & Data Querying

Insights & Analysis Report

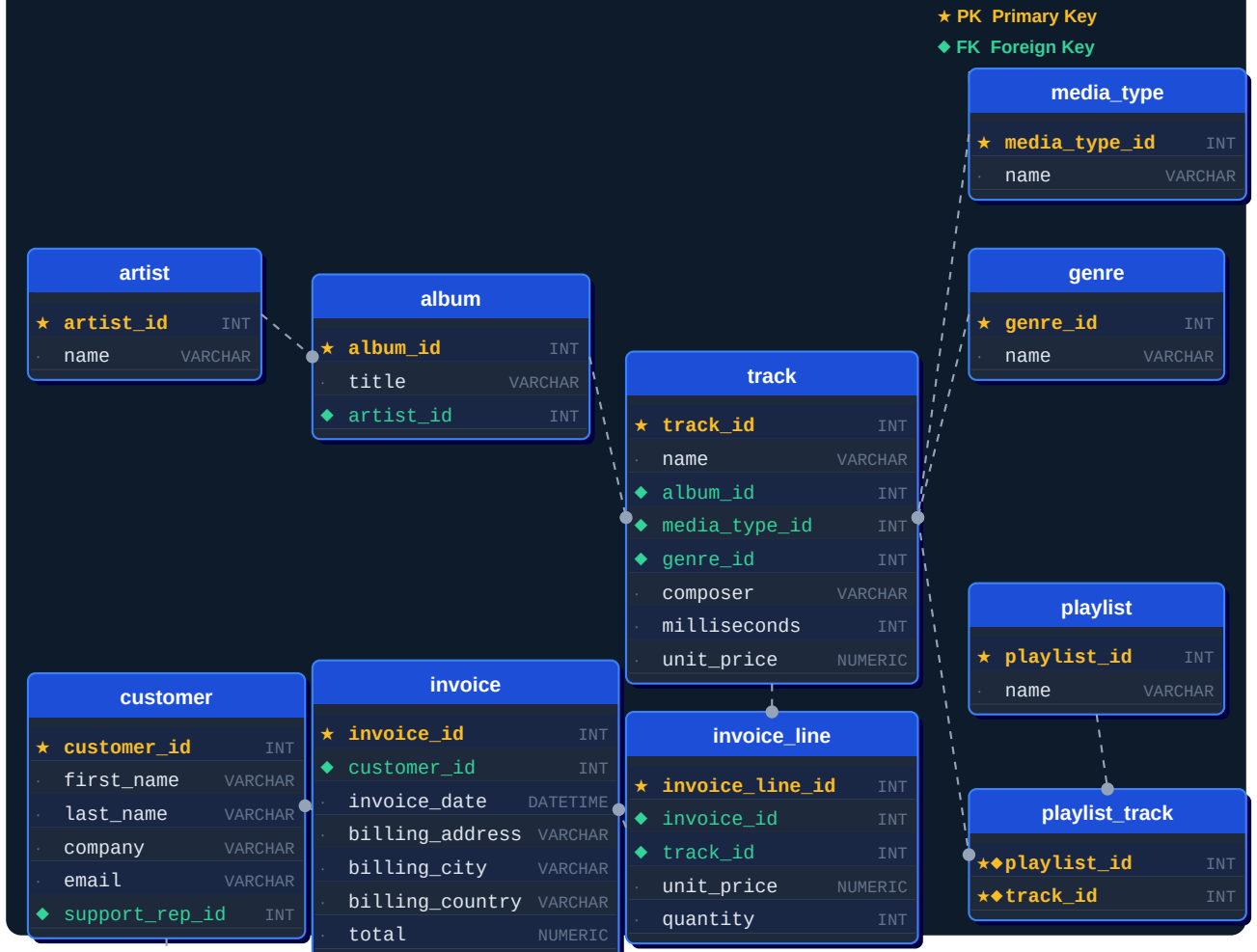
AnalystLab Africa · Data Analytics Internship · Batch B 2025

| Tool                    | Datasets               | Records                     | Sections              |
|-------------------------|------------------------|-----------------------------|-----------------------|
| PostgreSQL via Supabase | Chinook + Kaggle Sales | 3,503 tracks · 2,823 orders | 5 skill areas covered |

## 1. Chinook Database — Entity Relationship Diagram

The Chinook database models a digital music store. The diagram below shows all 11 tables, their primary keys (★ gold), foreign keys (◆ green), and the relationships between them.

## Chinook Database — Entity Relationship Diagram



| Table          | Description                      | Key Relationships                    |
|----------------|----------------------------------|--------------------------------------|
| artist         | Music artists / bands            | One artist → many albums             |
| album          | Album released by an artist      | One album → many tracks              |
| track          | Individual song or episode       | Links to genre, media_type, album    |
| genre          | Music genre classification       | One genre → many tracks              |
| media_type     | Format (MP3, AAC, etc.)          | One media type → many tracks         |
| invoice        | Customer purchase transaction    | One customer → many invoices         |
| invoice_line   | Line item within an invoice      | Links invoice to track purchased     |
| customer       | Store customer                   | Supported by one employee            |
| employee       | Support staff (self-referencing) | reports_to links to same table       |
| playlist       | Named collection of tracks       | Many-to-many with track              |
| playlist_track | Junction: playlist ↔ track       | Composite PK (playlist_id, track_id) |

## 2. Core SQL Queries

## 2.1 SELECT · WHERE · ORDER BY

Business question: Which US orders exceeded \$5,000 in sales value?

```
SELECT "CUSTOMERNAME", "ORDERNUMBER", "SALES", "COUNTRY"
FROM sales_data
WHERE "COUNTRY" = 'USA'
AND "SALES" > 5000
ORDER BY "SALES" DESC;
```

## ■ Result

CUSTOMERNAME SALES

|                           |           |
|---------------------------|-----------|
| The Sharp Gifts Warehouse | 14,082.80 |
| Anna's Decorations, Ltd   | 11,753.26 |
| Muscle Machine Inc        | 11,731.89 |

|                    |           |
|--------------------|-----------|
| Muscle Machine Inc | 11,731.89 |
|--------------------|-----------|

**Key Insight**  
The Sharp Gifts Warehouse placed the highest single-order value in the US at \$14,082.80 — over \$2,000 more than the next closest order. High-value accounts like this warrant priority customer service and loyalty programmes.

The Sharp Gifts Warehouse placed the highest single-order value in the US at \$14,082.80 — over \$2,000 more than the next closest order. High-value accounts like this warrant priority customer service and loyalty programmes.

## 2.2 GROUP BY · HAVING · SUM

Business question: Which customers generated more than \$50,000 in total revenue?

```
SELECT "CUSTOMERNAME",  
SUM("SALES") AS total_sales  
FROM sales_data  
GROUP BY "CUSTOMERNAME"  
HAVING SUM("SALES") > 50000  
ORDER BY total_sales DESC;
```

## ■ Result

CUSTOMERNAME TOTAL\_SALES

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

Euro Shopping Channel 912,294.11

Mini Gifts Distributors Ltd. 654,858.06

Australian Collectors, Co. 200,995.41

```
... (additional customers above threshold)
```

### Key Insight

Euro Shopping Channel dominates at \$912,294.11 — 39% above the second-highest account. HAVING filters aggregated groups after GROUP BY runs, unlike WHERE which filters individual rows before aggregation. This distinction is critical when writing aggregate-based business queries.

### 2.3 COUNT · AVG — Order Frequency Analysis

Business question: How many orders has each customer placed and what is their average order value?

```
SELECT "CUSTOMERNAME",  
COUNT(*) AS num_orders,  
AVG("SALES") AS avg_order_value  
FROM sales_data  
GROUP BY "CUSTOMERNAME"  
ORDER BY num_orders DESC;
```

## ■ Result

CUSTOMERNAME NUM\_ORDERS AVG\_ORDER\_VALUE

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464

Euro Shopping Channel 259 3,522.37

Mini Gifts Distributors 180 3,638.10

Super Scale Inc. 17 4,674.83 ← highest avg

### Key Insight

Euro Shopping Channel leads in order volume (259 orders) but its average order value (\$3,522) is lower than many smaller customers. Super Scale Inc. places only 17 orders yet averages \$4,674 each — revealing two distinct buying patterns that should drive different sales strategies.

## 3. Advanced SQL Concepts

### 3.1 – 3.3 JOIN Types (INNER · LEFT · RIGHT)

All three join types were applied between the customer and invoice tables in Chinook to understand data coverage and referential integrity.

| Join Type  | SQL Pattern                                                                 | Rows Returned | What It Shows                      |
|------------|-----------------------------------------------------------------------------|---------------|------------------------------------|
| INNER JOIN | FROM customer c<br>INNER JOIN invoice i<br>ON c.customer_id = i.customer_id | 412           | Only customers with invoices       |
| LEFT JOIN  | FROM customer c<br>LEFT JOIN invoice i<br>ON c.customer_id = i.customer_id  | 412           | All customers — NULL if no invoice |
| RIGHT JOIN | FROM customer c<br>RIGHT JOIN invoice i<br>ON c.customer_id = i.customer_id | 412           | All invoices — NULL if no customer |

#### Key Insight

All three join types returned exactly 412 rows — confirming perfect referential integrity in the Chinook dataset. Every customer has at least one invoice and every invoice belongs to a valid customer. In production databases, LEFT and RIGHT joins often reveal data quality gaps worth investigating.

### 3.4 Subquery — Customers Above Average Spend

Business question: Which customers spend more than the average Chinook customer?

```
SELECT c.first_name, c.last_name,
SUM(i.total) AS total_spent
FROM customer c
JOIN invoice i ON c.customer_id = i.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING SUM(i.total) > (
-- Subquery computes the average across all customers
SELECT AVG(customer_total)
FROM (SELECT SUM(total) AS customer_total
FROM invoice
GROUP BY customer_id) sub
)
ORDER BY total_spent DESC;
```

## ■ Result

22 of 59 customers (37%) spend above the database average.

FIRST\_NAME LAST\_NAME TOTAL\_SPENT

[illegible]

Helena Holy 49.62

Richard Cunningham 47.62

Luis Rojas 46.62

### Key Insight

The nested subquery first aggregates each customer's total spend, then averages those totals. This two-level aggregation is necessary because a simple `AVG(total)` would average individual invoice amounts, not customer-level totals — a common and important distinction.

### 3.5 – 3.6 ROW\_NUMBER vs RANK — Handling Ties

Both window functions rank customers by total spend, but they handle tied values differently.

```
-- ROW_NUMBER: always unique, no gaps

SELECT c.first_name, c.last_name,
SUM(i.total) AS total_spent,
ROW_NUMBER() OVER (ORDER BY SUM(i.total) DESC) AS row_num,
RANK() OVER (ORDER BY SUM(i.total) DESC) AS rank
FROM customer c
JOIN invoice i ON c.customer_id = i.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY rank;
```





## 4. Business Problem Solving

#### 4.1 Top Artists by Revenue (Chinook)

```
SELECT ar.name AS artist,
SUM(il.unit_price * il.quantity) AS revenue
FROM invoice_line il
JOIN track t ON il.track_id = t.track_id
JOIN album al ON t.album_id = al.album_id
JOIN artist ar ON al.artist_id = ar.artist_id
GROUP BY ar.name
ORDER BY revenue DESC;
```

## ■ Result

```
ARTIST REVENUE  
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■  
  
Iron Maiden 138.60  
  
U2 105.93  
  
Metallica 90.09  
  
Led Zeppelin 86.13  
  
Lost (TV Show) 81.59 ← TV show in music DB  
  
The Office (TV) 49.75 ← TV show in music DB
```



























Iron Maiden 138.60

U2 105.93

Metallica 90.09

Led Zeppelin 86.13

Lost (TV Show) 81.59 ← TV show in music DB

The Office (TV) 49.75 ← TV show in music DB

### Key Insight

TV shows "Lost" and "The Office" appear in the top 10 because Chinook stores all media types — music, TV episodes, and podcasts — under the same artist/album/track schema. This is a real-world schema design pattern: a generic media catalogue rather than a music-only database. The four-table JOIN chain (invoice\_line → track → album → artist) is the core revenue query.

TV shows "Lost" and "The Office" appear in the top 10 because Chinook stores all media types — music, TV episodes, and podcasts — under the same artist/album/track schema. This is a real-world schema design pattern: a generic media catalogue rather than a music-only database. The four-table JOIN chain (invoice\_line → track → album → artist) is the core revenue query.

#### 4.2 Top Product Lines by Revenue (sales\_data)

```
SELECT "PRODUCTLINE",  
SUM("SALES") AS total_sales  
FROM sales_data  
GROUP BY "PRODUCTLINE"  
ORDER BY total_sales DESC;
```

## ■ Result

```
PRODUCTLINE TOTAL_SALES
```

| Product Line     | Total Sales                        |
|------------------|------------------------------------|
| Classic Cars     | 3,919,615.66 ← more than double #2 |
| Vintage Cars     | 1,903,150.84                       |
| Motorcycles      | 1,166,388.34                       |
| Planes           | 975,003.57                         |
| Ships            | 714,437.13                         |
| Trucks and Buses | 695,354.08                         |
| Trains           | 226,243.47 ← weakest performer     |

### Key Insight

Classic Cars generates more than double Vintage Cars. Business implication: prioritise Classic Cars in inventory planning, marketing spend, and supplier negotiations. Trains (\$226K) may warrant a strategic review — either growth investment or gradual phase-out.

### 4.3 – 4.4 Monthly Revenue Trends

```
-- sales_data: real-world seasonality pattern
SELECT DATE_TRUNC('month', TO_DATE("ORDERDATE", 'MM/DD/YYYY')) AS month,
SUM("SALES") AS monthly_revenue
FROM sales_data
GROUP BY DATE_TRUNC('month', TO_DATE("ORDERDATE", 'MM/DD/YYYY'))
ORDER BY month;
```

| Dataset    | Month    | Revenue          | Observation                      |
|------------|----------|------------------|----------------------------------|
| Chinook    | Jan–Dec  | ~\$37.62 / month | Completely flat — synthetic data |
| sales_data | Oct 2003 | \$551,794.52     | Elevated — pre-holiday buildup   |
| sales_data | Nov 2003 | \$1,029,837.66   | ★ Peak — November spike          |
| sales_data | Oct 2004 | \$552,924.25     | Pattern repeats year-on-year     |
| sales_data | Nov 2004 | \$1,089,048.01   | ★ Peak — even higher than 2003   |

Chinook revenue is eerily flat at ~\$37.62/month — a hallmark of synthetic data. `sales_data` by contrast shows genuine Q4 seasonality: November spikes above \$1M both years, consistent with pre-holiday toy and collectibles buying. Business action: front-load inventory and staffing in September–October to capture this seasonal demand.

```
SELECT c.first_name, c.last_name,
MIN(i.invoice_date) AS first_purchase,
MAX(i.invoice_date) AS most_recent_purchase,
COUNT(i.invoice_id) AS total_purchases
FROM customer c
JOIN invoice i ON c.customer_id = i.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_purchases DESC;
```

## NAME FIRST\_PURCHASE LAST\_PURCHASE PURCHASES

[illegible]

Helena Holy 2021-07-11 2025-11-13 7

Richard Cunningham 2021-11-07 2025-04-05 7

```
... (55 more customers with exactly 7 purchases)
```

Puja Srivastava 2021-04-05 2024-05-30 6 ← only exception

Summary: 58 of 59 customers = exactly 7 purchases

A uniform purchase count of exactly 7 across 58 of 59 customers is statistically impossible in real data. Combined with the flat monthly revenue, this definitively confirms Chinook's invoice history is machine-generated with a fixed pattern. Always validate dataset authenticity before drawing business conclusions — this is a fundamental data quality check.

## 5. Query Optimisation

### 5.1 Baseline — EXPLAIN ANALYZE Before Indexing

The target query joins four tables to calculate revenue per artist. EXPLAIN ANALYZE was run first to capture the baseline execution plan.

```
EXPLAIN ANALYZE
SELECT ar.name,
SUM(il.unit_price * il.quantity) AS revenue
FROM invoice_line il
JOIN track t ON il.track_id = t.track_id
JOIN album al ON t.album_id = al.album_id
JOIN artist ar ON al.artist_id = ar.artist_id
GROUP BY ar.name
ORDER BY revenue DESC;
```

#### ■ Result

Planning Time: 4.088 ms  
Execution Time: 4.405 ms  
Scan strategy: Sequential Scans on all four tables

### 5.2 Creating Indexes on Join Columns

```
-- Index the FK columns used in each JOIN condition
CREATE INDEX IF NOT EXISTS idx_invoice_line_track_id
ON invoice_line (track_id);

CREATE INDEX IF NOT EXISTS idx_track_album_id
ON track (album_id);

CREATE INDEX IF NOT EXISTS idx_album_artist_id
ON album (artist_id);
```

### 5.3 After Indexing — EXPLAIN ANALYZE Results

#### ■ Result

Planning Time: 1.625 ms (reduced – planner benefits from index metadata)  
Execution Time: 4.613 ms (slightly slower – indexes not used at runtime)  
Scan strategy: Sequential Scans still chosen on all four tables

| Metric        | Before Indexing | After Indexing | Change       |
|---------------|-----------------|----------------|--------------|
| Planning Time | 4.088 ms        | 1.625 ms       | ▼ 60% faster |

|                |          |          |                              |
|----------------|----------|----------|------------------------------|
| Execution Time | 4.405 ms | 4.613 ms | ▲ Slightly slower            |
| Scan Type      | Seq Scan | Seq Scan | Unchanged                    |
| Index Used?    | N/A      | No       | Planner skipped indexes      |
| Verdict        | —        | —        | No net benefit at this scale |

### Key Insight

PostgreSQL's query planner correctly chose sequential scans over indexes. At small table sizes (the largest Chinook table has only 3,503 rows), reading every row is faster than the overhead of index lookups. The planner's cost model determined that a full scan is cheaper — this is correct and expected behaviour. Indexes provide meaningful benefit typically above 10,000+ rows, when a query retrieves only a small fraction of a large table.

## 6. Summary of Key Insights

- 1 Revenue concentration:** Euro Shopping Channel accounts for \$912,294.11 in sales\_data — 39% above the second-highest account — driven by order volume (259 orders) rather than high order value. Chinook customer spend, by contrast, is distributed much more evenly.
- 2 Referential integrity confirmed:** INNER, LEFT, and RIGHT joins between customer and invoice all returned exactly 412 rows, confirming no orphaned records exist on either side of the Chinook dataset — a clean bill of health for data quality.
- 3 37% above average:** 22 of 59 Chinook customers (37%) spend above the per-customer average, identified using a nested subquery. Correct two-level aggregation — aggregate first, then average — is essential for this type of segmentation analysis.
- 4 RANK vs ROW\_NUMBER:** A 29-way tie at \$37.62 all received RANK 29, with the next customer jumping directly to rank 59. ROW\_NUMBER assigned unique values 29–57 to those same customers. Choose based on whether ties should share a rank or be uniquely ordered.
- 5 PARTITION BY resets per group:** Track rankings reset to 1 at every genre boundary across all 25 genres. PARTITION BY is the instruction that creates independent windows — without it, all 3,503 tracks would rank in a single global list.
- 6 Chinook is synthetic:** Flat monthly revenue (~\$37.62/month for 5 years) and exactly 7 purchases per customer for 58 of 59 users are statistical impossibilities in real data. Always validate dataset authenticity before drawing business conclusions.
- 7 Genuine seasonality in sales\_data:** November revenue exceeded \$1,000,000 in both 2003 and 2004 — consistent with pre-holiday toy and collectibles demand. This pattern has direct business implications for inventory planning, staffing, and marketing timing.
- 8 Classic Cars dominates:** At \$3,919,615.66, Classic Cars generates more than double the second-highest product line. Iron Maiden leads Chinook artist revenue at \$138.60. TV shows "Lost" and "The Office" appearing in the top 10 artists reveal Chinook's multi-media schema.
- 9 Indexing at small scale adds no benefit:** The PostgreSQL planner correctly ignored all three indexes and continued using sequential scans. Index benefit emerges at scale — typically tables of 10,000+ rows where queries retrieve a small fraction of total data.